

BFTRAND: Low-latency Random Number Provider for BFT Smart Contracts

Abstract—Random numbers play a crucial role in various decentralized applications (dApps), including decentralized finance (DeFi) and non-fungible tokens (NFTs). However, generating these numbers presents challenges due to the deterministic and decentralized nature of blockchains, potentially compromising smart contract security and the entire ecosystem. Previous research has examined commit-execute schemes, such as Oracles, which face drawbacks such as increased transaction fees, extended processing times, and on-chain storage bloat.

This paper proposes a novel random number provider (RNP) protocol for smart contracts that eliminates the reliance on commit-execute schemes. We systematically analyze random number attacks on smart contracts, identify potential attacks, including Post-reveal Undo Attacks (PUAs) that allow adversaries to reverse contract operations if the generated randomness is unfavorable, and discuss the security requirements. Our protocol addresses these attacks by (1) incorporating distributed random beacons (DRBs) into the consensus, bridging the semantic gap between DRB and consensus, and (2) thoroughly analyzing and classifying four types of PUA, offering appropriate mitigations. We also present a formal proof illustrating the security of our protocol.

Experimental results demonstrate that our protocol provides low-latency responses to smart contract randomness requests, effectively protects against random number attacks, reduces fee costs for random number requests by at least 89%, and generates 76.4% less on-chain data compared to existing commit-execute solutions. Our research bridges the security and semantic gap between DRB protocols and consensus mechanisms, enhancing secure and efficient random number utilization in decentralized applications and smart contracts, and contributing to the development of more reliable and resilient systems.

I. INTRODUCTION

Smart contracts form the basis of decentralized applications (dApps) ecosystems, particularly in the decentralized finance (DeFi) and non-fungible tokens (NFTs) sectors [45]. These applications often rely on public random number providers (RNP) to perform tasks such as determining the winners, shuffle cards [45], and ensuring the uniqueness, rarity, and equitable distribution of NFTs [36], [73], [78]. Additionally, legal toolkits use random jurors to facilitate efficient dispute resolution [70]. However, generating secure random numbers in smart contracts poses challenges due to the deterministic and decentralized nature of blockchain systems. These characteristics allow adversaries to compute or bias random numbers, leading to various random number attacks [40], [58], [87], [89]. For example, the dApp EOSBet encountered a nonce attack in 2018, resulting in the theft of tokens [41]. The attackers manipulated the RNP, allowing them to control the outcomes of the game.

Secure RNP protocols must generate random values resistant to random number attacks [25]. Some existing schemes, such as distributed random beacon (DRB) [2], [19], [30], [35], [42], [54], [60], [84], [85], [90], the verifiable delay function efficiency, making a significant contribution to the execution

(a) Commit-execute RNP

(b) BFTRAND RNP

Figure 1: Comparison between commit-execute RNP and BFTRAND RNP. T_{commit} and T_{execute} represent transactions to commit function parameters and execute the function, respectively. Solid lines indicate one consensus round. Commit-execute RNP (a) requires two transactions in two distinct consensus rounds to supply the requested random number, while BFTRAND (b) fulfills one request within one consensus round. RNP components are shaded in gray

(VDF) [6], [49], and verifiable random function (VRF) [4], [34], are promising protocols to produce secure random numbers; however, to use the numbers in smart contracts generated by those protocols, a commit-execute RNP mechanism must be utilized, as illustrated in Figure 1a. These methods involve recording user parameters in the transaction T_{commit} during a block interval and generating universally accessible randomness. A second transaction T_{execute} provides random numbers in the subsequent block time, preventing attackers from predicting the random number in advance. Although this approach improves RNP security, it results in high transaction fees, extended processing time, and increased on-chain storage. Furthermore, these schemes do not take into account the specific security requirements and environmental constraints inherent in blockchain systems, such as the need to address the risk of post-reveal undo attacks (PUAs). In these areas, the current RNP approaches reveal their inadequacy for application in the context of smart contract execution, causing the need for a new approach.

BFTRAND Protocol: We present a new low-latency RNP protocol, called BFTRAND, which is specifically designed to deliver random numbers to smart contracts within a single consensus round, as illustrated in Figure 1b. BFTRAND addresses the challenges associated with existing commit-execute RNP schemes, such as Chainlink VRF [34] and Drand [44], while offering a secure and efficient approach to generate random numbers for smart contracts.

By incorporating DRBs into the consensus process, BFTRAND introduces a beacon for each block and quickly generates random numbers for smart contracts upon receiving requests. This advanced protocol improves security and

of smart contracts and blockchain-based applications.

BFTRAND generates low-latency random numbers during the execution of smart contracts, resulting in reduced response times for random number requests. Our protocol includes robust security measures to protect against random number attacks, effectively addressing the risks associated with post-reveal undo attacks (PUA), which allow adversaries to reverse critical operations if the randomness outcome is unfavorable (SIV).

Challenges: Our proposed protocol tackles two main challenges: seamlessly integrating DRB without compromising consensus efficiency and security and mitigating PUA risks. We acknowledge the following security challenges to maintain high standards of security and develop comprehensive definitions and properties to address them.

Insufficiency of DRB. The innovative direct request-response paradigm of the BFTRAND protocol significantly enhances the effectiveness of randomness provision, deviating from the prevalent commit-execute framework seen in current RNP protocols. However, the semantic discrepancy between DRBs and consensus may reduce the efficacy of DRBs' security features upon integration, highlighting the need for a thorough approach to harmonize these elements. DRBs inherently possess security properties such as pseudo-randomness and uniqueness [51], [55], [82]. Our analysis indicates that incorporating DRBs into blockchain architectures to provide random numbers for smart contracts introduces new security and performance challenges. These challenges arise from the conflicting security assumptions and environmental constraints inherent in blockchain systems. For example, DRB processes occur in sequential phases, whereas the foundational consensus of our protocol, Byzantine Fault Tolerance (BFT), operates in consensus rounds. Each round consists of multiple views that share the same round context. Careful management of views within BFT rounds is necessary to prevent adversaries from predicting random numbers across views (A).

Post-reveal Undo Attack (PUA). Integrating DRBs into BFTRAND does not eliminate all random number attacks. Our thorough analysis reveals that including random numbers in smart contracts at runtime exposes a previously unexplored attack surface, termed Post-reveal Undo Attack (PUA).

II. BACKGROUND

This work focuses on a blockchain system based on Practical Byzantine Fault Tolerant (PBFT) with smart contracts consisting of n nodes, $\mathcal{N} = N_0; \dots; N_{n-1}$. The set of corrupted nodes controlled by the adversary, denoted by $\mathcal{A}$, is represented as $\mathcal{C}$. Let $D; R : \mathbb{N} \rightarrow \mathbb{N}$ denote polynomial-time functions, with both $D(\lambda)$ and $R(\lambda)$ bounded by a polynomial in λ . Furthermore, let $F : \text{Dom} \rightarrow \text{Ran}$ be a function with domain Dom and range Ran , where Dom and Ran are sets with sizes $2^{D(\lambda)}$ and $2^{R(\lambda)}$, respectively. We also assume the existence of a pseudorandom function (denoted $\text{PRF} : \text{Ran} \rightarrow \text{Ran}'$).

A. Blockchain

This section provides a brief overview of fundamental concepts and technologies relevant to our work, including smart contracts, blockchain consensus mechanisms, and random beacons.

1) *Smart Contracts*: Smart contracts are self-executing agreements with terms encoded in the program. They are used primarily in decentralized applications (dApps) built on platforms such as Ethereum [28] and NEO [72]. These contracts automate complex business logic and enforce contractual terms without intermediaries. They are immutable and transparent, ensuring a high level of trust and security.

2) *Byzantine Fault Tolerance*: Byzantine Fault Tolerance (BFT) represents a class of consensus algorithms, including PBFT [32] and its variants [3], [8], [12], [15]–[17], [29], [31], [37], [61], [64], [66], [69], [72], [92], [93], [95], which address the Byzantine General Problem in distributed systems and provide instant finality [65]. Currently, 17 of the top 80 blockchain projects use the BFT consensus [38], including Theta [74], Neo [72], Algorand [5], and EOS [46].

BFT consensus enables distributed systems to achieve consensus despite faulty nodes exhibiting unpredictable behavior. It operates in rounds, with each round having one or more views [32]. At the end of each round, a new block is formed that represents the consensus of the network. An $(f; t; n)$ secure BFT protocol with n nodes can tolerate up to t faulty nodes and is secure for any probabilistic polynomial-time (PPT) attacker $\mathcal{A}$ if $0 \leq f \leq t < n/3$ for asynchronous BFT [32]. A BFT consensus protocol adheres to consistency and liveness properties [31].

For clarity, we will now provide a concise explanation of PBFT [32], which is a commonly used variant of BFT. PBFT operates through consensus rounds, with each round led by a designated leader. The leaders are chosen in a round-robin fashion. Specifically, the function $\text{Leader}(b; v; n) \rightarrow I$ is defined to select the leader index I for round b of view v , where n represents the number of participants. In other words, given the values of view v , round b and the total number of participants n , the function Leader outputs the leader index I . The PBFT algorithm consists of the following phases [32]: *Prepare*. The leader receives user requests and transactions and starts the consensus process by sending a “prepare” message to the consensus nodes that are not the leader (replicas).

Response. Upon receiving the “prepare” message, a node sends a “respond” message to all other nodes, including the leader. *Commit*. A consensus node sends a “commit” message after collecting more than $2t+1$ “respond” messages. *Finalize*. Once a node has obtained over $2t+1$ “commit” messages, consensus is achieved and the node provides the execution result to users.

Viewchange: This mechanism is a critical aspect of the PBFT algorithm, which is triggered by a “viewchange” message to ensure fault tolerance in distributed systems. If a node encounters a timeout during any of the aforementioned phases, it constructs and broadcasts a view change message to request a fresh view. If more than $2t$ view change messages are received, a new leader will restart the entire process with a new view.

B. Distributed Random Beacons

Distributed Random Beacons (DRBs) are a well-established class of RNPs that generate random numbers, called beacons within the DRB context, through a decentralized process. This approach ensures that no single entity can manipulate or predict the generated values. Various DRB protocols have been proposed in the literature, including Drand [44], DFINITY [2], RandChain [54], RandPiper [19], DDH-DVRF and GLOW-DVRF [51], RandHound and RandHerd [90]. These values are vital to ensure the security and robustness of numerous applications, particularly smart contracts, within the rapidly evolving blockchain ecosystem. The feasibility of constructing a DRB protocol has been demonstrated in the literature [51], [55], [82]. In this paper, we will utilize the DRB scheme proposed in [51], and we provide the formal definition below:

- 1) **Setup**($1^\lambda, k, n$). The nodes in $\mathcal{N}$ execute this interactive protocol to establish a random beacon committee. The protocol produces a global public key pk , a list of public keys $\{pk_0, \dots, pk_{n-1}\}$, and a list of secret keys $\{sk_0, \dots, sk_{n-1}\}$.
- 2) **Partial**($\text{st}_{rn-1}, sk_i, pk$). Given a state $\text{st}_{rn-1} \in \text{Dom}$ of round $rn-1$, a secret key sk_i , and a global public key pk , the algorithm computes a partial beacon σ_i and a proof π_i . The output is (i, σ_i, π_i) .
- 3) **Comb**($\text{st}_{rn-1}, \mathcal{E}, pk$). Given a state $\text{st}_{rn-1} \in \text{Dom}$, a set $\mathcal{E} = \{\sigma_i, \pi_i\}_{i \in I}$ of partial values and proofs from $|I| \geq k$ different nodes and a global public key pk , this algorithm produces a beacon σ_i and its proof π_i , or $\perp$.
- 4) **Verify**($\text{st}_{rn-1}, \sigma_{rn}, \pi_{rn}, pk$). Given an input state $\text{st}_{rn-1} \in \text{Dom}$, a pair consisting of a beacon σ_{rn} and proof π_{rn} , and a public key pk , this algorithm verifies the validity of σ_{rn} . It outputs 0 if the beacon is valid and 1 otherwise.
- 5) **UpdState**($\text{st}_{rn-1}, \sigma_{rn}, \pi_{rn}, pk$). Given the current state st_{rn-1} , a beacon $\sigma_{rn} \in \text{Ran}$ and its proof π_{rn} generated at the end of the round $rn-1$, and a global public key pk , the algorithm outputs the updated state st_{rn} for the round rn , or $\perp$.

Security Requirements. *Pseudorandomness*: ensures that the generated beacon cannot be distinguished from a uniformly chosen value in the presence of active adversaries, denoting unpredictability and bias-resistance; *Uniqueness*: requires the generated random beacon is unique with the fixed input even if the attacker can access the secret keys of honest parties.

III. PROBLEM FORMULATION

In this section, we provide a comprehensive overview of our BFTRAND protocol (§III-A) and examine the security assumptions (§III-B). Subsequently, we briefly explain the

required security properties (§III-C). In the full version [9], we provide an explanation of random number attacks.

A. Overview of BFTRAND

BFTRAND aims to provide a low-latency secure random number service for smart contracts. It generates a random beacon for each block and derives multiple random numbers for smart contracts from the generated beacon.

In BFTRAND, a *proposal* refers to a candidate block that contains a list of transactions from the leader. This proposal will be confirmed at the end of the consensus process if more than $2t$ nodes agree. For the consensus round $b > 0$ in view $v \geq 0$, the proposal is denoted as $\mathcal{L}_{b,v}$. To simplify our notation, we use b to represent both the consensus round and the corresponding block. BFTRAND is divided into three main sections: Initialization, Beacon Request, and Randomness Request, each describing the necessary steps to achieve the desired outcomes.

Initialization: Before executing BFTRAND with DRB, it is crucial to initialize the context of BFTRAND to establish the execution environment. During this phase, the [DRB.Setup](#) function generates public and private keys for the participants. Additionally, an empty dictionary of random beacons, denoted by $RB = \{\}$, is initialized to store random beacons generated indexed by unique identifiers.

Beacon Request: BFTRAND generates a random beacon for each block after the point at which new transactions can no longer be created or the block proposal cannot be altered. This beacon acts as a random seed to produce random numbers within the corresponding block. A beacon request round begins when a consensus leader sends a tuple $(\text{Beacon}; b; v)$, where b denotes the block number and v represents the current view number. A new random beacon, rb , is generated by DRB execution at consensus in the BFT among participants. The beacon is then stored in the dictionary with the index b as $RB[b] = rb$.

Randomness Request: A smart contract requests randomness by sending a tuple $(\text{Randomness}; b; m; \mathcal{T})$. The protocol retrieves the corresponding random beacon from the dictionary using the index b , denoted as $rb = RB[b]$. Next, m random numbers, $(r_1; \dots; r_m)$, are calculated. The calculated random numbers are then returned to the requesting smart contract.

B. Security Assumptions and Threat Model

This section presents the security assumptions and threat model that form the foundation of our research on Random Number Generation (RNP) in smart contracts. In our analysis, we consider an attacker, denoted as $\mathcal{A}$, who has control over up to $t < n/3$ nodes in the blockchain network. These controlled nodes may exhibit Byzantine behavior, allowing the attacker to manipulate their actions arbitrarily. We assume that the attacker is static, meaning that the nodes under their control are determined before the consensus protocol is initiated. This allows us to focus on the core security properties of our RNP protocol. The attacker has complete control over the faulty nodes, including their network connections and operating

systems. Consequently, the attacker has access to all secret keys of the compromised nodes, allowing them to counterfeit any message on their behalf. The attacker is also able to observe the blockchain network, giving them knowledge of all transactions and consensus messages. If the leader node is corrupted, the attacker can propose a transaction list that benefits them. Nodes that remain uncorrupted throughout the consensus process are considered honest and strictly adhere to the consensus protocol. These nodes maintain stable network connections, possess sufficient computing power, and have enough storage capacity to process blockchain transactions. There is no collusion between honest and corrupted nodes, and when an honest node receives a valid transaction, it promptly broadcasts the transaction to all other consensus nodes.

C. Security Properties

The random number security analysis can be found in the full version [9]. To ensure the security of our RNP protocol and effectively counteract random number attacks on smart contracts, we have established a set of properties to which BFTRAND must adhere. Let Π be the RNP protocol, $\mathcal{A}$ be an adversary, and $\mathcal{U}$ be a uniform random function. Let $\mathcal{D}$ be a distinguisher that interacts with $\mathcal{A}$ and a protocol/function, then returns 1 if it believes that it interacted with Π and 0 otherwise.

Pseudorandomness. A secure RNP should achieve *pseudorandomness*, which requires the generated random numbers to be indistinguishable from truly random numbers within practical time and computational resource constraints. This property guarantees that an attacker cannot predict the random numbers for his transaction, thwarting precomputation attacks.

Uniqueness. The *uniqueness* is a crucial characteristic of BFTRAND, which stipulates that only one valid and deterministic random number can be generated for any given input. Enforcing uniqueness prevents potential adversaries from manipulating multiple valid random numbers for identical inputs or exploiting race conditions. Consequently, uniqueness effectively mitigates replay attacks and obstructs validator collusion attacks, thus enhancing RNP security.

Availability. The availability ensures the continuous generation of random numbers by the RNP for each smart contract request. Protect against potential DoS attacks or other disruptions that aim to compromise the functionality of the blockchain system and its applications, enhancing the overall security and reliability of the system.

Irreversibility. The property of *irreversibility* ensures that the execution of a transaction does not fail unintentionally. This security prerequisite significantly reduces the likelihood that adversaries exploit PUAs to compromise “critical operations” and reverse the transaction outcome. Consequently, it provides a solid foundation for the secure execution of smart contracts that require random numbers.

IV. POST-REVEAL UNDO ATTACK

In this section, we discuss PUA in further detail. We investigate four distinct types of PUAs: **Contract PUAs**, **Fallback**

```

1 function MintNFT(from, target, amount)
2   CheckWitness(from)
3   Require(amount == 1)
4   ❶ Transfer(from, this, amount)
5   rarity = GetRandom(1) % 2
6   if (rarity == 1){
7     // Cost 0.5 GAS
8     blindBox = RareBlindBox(target, rarity)
9   }
10  ❷ }else{
11    // Cost 0.6 GAS
12    blindBox = CommonBlindBox(target, rarity)
13  }
14  Mint(target, blindBox)
15  ❸ Transfer(this, target, 1, {blindBox})
16  StoreBlindBox(blindBox, rarity)
17  return blindBox
18 end function
19 function GetRarity(blindBox)
20   return StoreBlindBox[blindBox]
21 end function

```

Figure 2: Pseudocode representation of the two core functions within the vulnerable BlindBox, the victim contract, susceptible to PUA attacks. ❶ The “MintNFT” function accepts parameters (from, target, amount), where ‘from’ denotes the user address, ‘target’ represents the address receiving the NFT token, and ‘amount’ specifies the number of tokens transferred. “MintNFT” processes a transfer of one token from a user, ❷ employs random numbers to ascertain the rarity of the newly minted NFT, and ❸ transfers the NFT to the intended recipient. The “GetRarity” function retrieves the rarity of a specific NFT. Note that the GAS in the code is a dummy token utilized solely for contract demonstration.

PUAs, Fee PUAs (which arise from smart contract designs), and **Script PUAs** (specific to script-based blockchains). To demonstrate how these PUAs exploit vulnerable contracts, we create a representative BlindBox contract as an illustrative example.

A Victim Contract - BlindBox Contract. Figure 2 presents the pseudocode of a BlindBox contract designed to demonstrate the effects of various PUA approaches. This contract allows users to deposit a token, mint a corresponding NFT, and transfer it to a specified recipient. The rarity of the NFT depends on a random number acquired during run-time through the “GetRandom” function. The “Critical Operation” in the code block ❶ represents a user who submits a one token bid for an NFT. The random number in the code block ❷ determines the rarity of the minted NFT, while the code block ❸ transfers the minted NFT to the designated address. Adversaries exploit this contract by reversing the deposit operation in the code block ❶ if the minted blind box does not have a high rarity. This action contradicts the nonrefundable and nonexchangeable principles inherent in the blind box mechanism.

Contract PUA. The functionality of smart contracts is significantly enhanced by their interoperability, which allows a single transaction to invoke multiple smart contracts through inter-operation. Transactions are atomic, meaning that if any of the contracts involved in a transaction fails and reverts, all other executed contract calls will also be rolled back [94]. Attackers can exploit this interoperability by deploying a malicious

```

1 function ContractPUA(user)
2   ❹ blindBox = call NFT.MintNFT with (user, 1)
3   rarity = call NFT.GetRarity with (blindBox)
4   ❺ if rarity == 0 then
5     call revert();
6   end
7   return true
8 end function

```

Figure 3: Pseudocode representation of the Contract PUA malicious contract. NFT represents the BlindBox contract.

contract beforehand and invoking the victim contract within the malicious contract. The malicious contract then examines the results of the execution of the victim contract [14], [68]. Figure 3 presents the pseudocode of the malicious contract, which includes a method named “ContractPUA” that initiates the PUA. In line 2, the “MintNFT” method of the victim contract is called, and in line 3, the rarity of the minted NFT token is retrieved and checked in the code block ❺. The attacker invokes the victim contract in code block ❹, calling the “MintNFT” method of the BlindBox contract and minting an NFT with a random rarity. The attack contract then verifies the result against their expectations in the code block ❺. If the minted NFT token is not considered rare, the attacker can deliberately reverse execution, undoing Critical Operation (code block ❶ in Figure 2). Contract PUA differs from the pre-computation attack, as it does not compute random numbers.

Fallback PUA. The fallback function is a default mechanism in smart contracts that is executed when receiving a transaction without a specified function call or if the requested function does not exist [14]. This feature allows contracts to handle unexpected scenarios or implement custom logic for incoming transactions [68]. Attackers may exploit the fallback function to target victim contracts in the following way. By creating a malicious contract with a strategically designed fallback function, the attacker can influence the behavior of the victim contract during interactions with the malicious contract [53]. Consequently, the attacker can utilize the fallback function to verify the execution result of the target contract and potentially alter or reverse the intended outcome of the contract. This attack is known as a fallback PUA. Figure 4 illustrates a fallback PUA contract. If the “target” address in code block ❸ of Figure 2 matches the address of the malicious contract, the malicious contract is invoked when executing the code block ❸. The malicious contract can then assess the rarity of NFT in the code block ❺ [62]. By exploiting the fallback function mechanism, attackers can launch attacks that interfere with the standard operation of the victim contract and compromise its security [77].

Fee-based PUA. Transaction fees play a crucial role in incentivizing validators or miners to maintain the blockchain network. Attackers who cannot directly verify state changes may exploit fee-based side channels to execute PUAs without interacting with the targeted contracts. This strategy is called a fee-based PUA. Figure 5 illustrates an example of fee-based PUA. As mentioned above, the “MintNFT” function

```

1 function Fallback(from, amount, object)
2   rarity = call NFT.GetRarity with (blindBox)
3   ⑥ if rarity == 0 then
4     call revert();
5   end
6   return true
7 end function

```

Figure 4: Pseudocode representation of a Fallback PUA contract. NFT represents the vulnerable BlindBox contract.

```

1 function MintNFT(from, target, amount)
2   CheckWitness(from)
3   Require(amount == 1)
4   Transfer(from, this, amount)
5   rarity = GetRandom(1) % 2
6   if (rarity == 1){
7     // Cost 0.5 GAS
8     blindBox = RareBlindBox(target, rarity)
9   }else{
10    // Cost 0.6 GAS
11    ⑧ // revert if fee insufficient
12    blindBox = CommonBlindBox(target, rarity)
13  }
14  Mint(target, blindBox)
15  Transfer(this, target, 1, {blindBox})
16  StoreBlindBox(blindBox, rarity)
17  return blindBox
18 end function

```

Figure 5: Pseudocode representation of a Fee PUA. The code block in ⑦ consumes an additional 0.5 GAS, while the code block ⑧ consumes an additional 0.6 GAS. Code blocks ⑦ and ⑧ are marked as ② in Figure 2.

is a function of the BlindBox contract. Line 6 of the code examines the rarity, which is determined by a random number. If the rarity is 1, the code proceeds to block ⑦, creating a rare blind box and incurring a transaction fee of 0.5 GAS. Otherwise, it moves to block ⑧, generating a common blind box at the cost of 0.6 GAS. If the attacker is aware of the gas costs for both paths and intends to revert Critical Operation when ⑦ is selected, they can set the maximum transaction fee to a higher value, which is sufficient for executing block ⑦, but lower than the cost of block ⑧. Identifying fee-based PUAs is substantially more challenging compared to other PUA categories.

These three prevalent PUAs mentioned above are prevalent in many smart contract platforms and fundamentally arise from the core design principles of blockchain and smart contracts [14], [77], [94]. These principles comprise interoperability, fallback mechanisms, and gas fees, which are crucial components of the smart contract ecosystem [77]. In addition to these generalized PUAs, a distinct PUA has been identified, specifically unique to script-based smart contract platforms [39].

Script-based PUA. In script platforms, transactions are represented as scripts and executed within a virtual machine (e.g., Neo Virtual Machine [72]). Attackers can exploit this feature by adding additional checking logic to the transaction script after invoking the victim contract to verify and manipulate the execution results.

```

1 load "Blindbox Contract"
2 push amount
3 push target
4 push from
5 call "MintNFT" // push blindBox to the stack
6 call "GetRarity" //pop blindBox, push rarity
7 jmpz revert() // if rarity == 0, jump to revert() ⑨

```

Figure 6: Transaction script pseudocode representation of a Script PUA. The code block ⑨ is the attached verification logic.

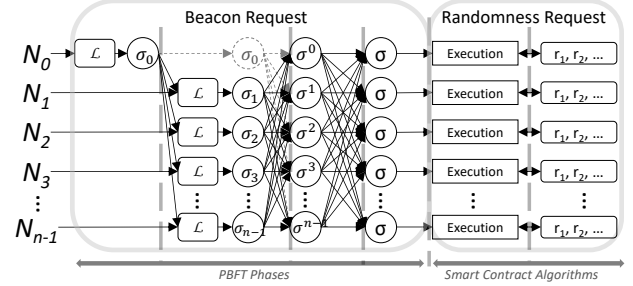


Figure 7: High-level Π_{BFTRAND} description for beacon request and for randomness request. $\mathcal{L}$ is the *proposal* from the leader for some view v of round b . $\{\sigma_i\}$ are the partial beacons, whereas $\{\sigma^i\}$ are the beacons aggregated by nodes, and σ is the final beacon. $\{r_i\}$ are random numbers. Execution entails running smart contracts with transaction inputs on a virtual machine. We omit the corresponding subscripts relating to the view v and round b in $\mathcal{L}$, σ_i , σ^i and σ .

Figure 6 presents the pseudocode for a transaction script. During the execution process, the virtual machine (VM) first loads and runs the victim contract (line 1), pushes the required parameters to the stack, and subsequently calls the “MintNFT” method (line 5). This method retrieves the parameters from the stack, processes them, and pushes the resulting output back onto the stack. After the “MintNFT” method has concluded, the script invokes “GetRarity” (line 6), which accesses the NFT token from the stack and pushes its rarity value. The script then examines the rarity and if it is deemed unsatisfactory, the attacker can intentionally reverse the transaction [59]. This attack exemplifies the security concerns associated with script-based smart contract platforms and emphasizes the importance of implementing robust security measures while developing and deploying smart contracts on these platforms.

V. BFTRAND PROTOCOL Π_{BFTRAND}

We introduce BFTRAND, a low-latency RNP protocol designated as Π_{BFTRAND} , meticulously crafted for BFT consensus. This versatile protocol can be adapted for various BFT-style consensus blockchain systems. As illustrated in Figure 7, the protocol functions in two primary stages: the beacon request phase, responsible for generating the block beacon, and the randomness request phase, providing random numbers for smart contracts.

We assume the existence of an efficient pseudo-random function called *PRF*. Let *PRF* be an efficient keyed function family, $\text{PRF} : \{\mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}\}$, where $\mathcal{K}$; $\mathcal{X}$ and $\mathcal{Y}$ are indexed with a security parameter λ . For a function F_k in *PRF* where

its key k is uniformly chosen from $\mathcal{K}$, the following holds for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}^{f(\cdot)} = 1] - \Pr[\mathcal{A}^{f'(\cdot)} = 1] \right| \leq \text{negl}(\lambda); \quad (1)$$

where f is a function uniformly chosen from the functions mapping from $\mathcal{X}$ to $\mathcal{Y}$. Looking ahead, we denote $\mathcal{X}$ as Ran , while $\mathcal{Y}$ as Ran' in our construction.

A. Beacon Request

To design the beacon request protocol for BFTRAND, we adopt the DRB scheme from [51]. Incorporating DRB directly into the consensus process and generating a beacon for each block may seem like an intuitive design choice. However, our analysis reveals a significant semantic gap between DRB and consensus. Without careful design, the system would be vulnerable to random number attacks, potentially compromising the security properties of both DRB and consensus and degrading consensus performance. We identify the following primary reasons for this gap.

First, the beacon must be generated at a specific time to prevent it from being exploited. Second, the threshold of the DRB differs from the Byzantine tolerance of the consensus and has distinct security assumptions. Improper initialization could result in validator collusion attacks or DoS attacks. Third, the DRB and consensus operate in separate rounds, and their direct integration could lead to precomputation or replay attacks.

BFTRAND is designed upon PBFT, and it requires seamless integration with DRB. To achieve this, the generation of beacons from DRB must be completed before the Finalize phase of PBFT to prevent any disruption of the consensus process. Additionally, the aggregated beacon should be broadcast along with the commit message. During the Prepare phase, the PBFT leader must also utilize DRB to generate the partial beacon and broadcast it with the message “prepare”. The other consensus nodes generate their partial beacons during the Response phase and transmit them with the response message. Failure to follow these phases will introduce additional consensus phases, compromising the overall performance of the consensus process.

The DRB generates a random beacon by aggregating the threshold of k or more authenticated partial beacons. To align the DRB threshold k with the Byzantine fault tolerance assumption of PBFT, we need to investigate the valid range for k . First, k must exceed the maximum PBFT Byzantine tolerance t . If $k \leq t$, an attacker $\mathcal{A}$ could prematurely aggregate the beacon, leading to precomputation attacks. Furthermore, k must not exceed $2t + 1$. Otherwise, DRB.Comb would require over $2t + 1$ partial beacons to aggregate the final beacon, thus demanding the cooperation of Byzantine nodes and potentially causing DoS attacks. Consequently, k must be within the range $(t; 2t + 1]$ to effectively mitigate both DoS and precomputation attacks.

The DRB operates in rounds, and each round rn generates a random beacon. Meanwhile, PBFT functions according to the consensus round b , and RNP requires only one random beacon per consensus round. Thus, the intuitive design assigns the consensus round b as the DRB round rn , that is, $rn \leftarrow$

b . However, integrating DRB with PBFT might introduce a security concern since PBFT consensus can create multiple views. PBFT switches views if consensus fails in the current view and restarts consensus in a new view. If DRB and PBFT employ the same number of rounds, different views, v and $v + 1$, of the same consensus round will share the same input state, that is, $\text{st}_{rn}^{b,v} = \text{st}_{rn}^{b,v+1}$, resulting in beacon leakage, which is a replay attack. Therefore, if the prior consensus changes the view at or after the PBFT Commit phase, the attacker $\mathcal{A}$ can compute the random beacon before the new view begins.

To mitigate this potential vulnerability, we propose modifying the assignment of the DRB round number rn . Instead of directly assigning rn as the consensus round b , we combine the consensus round b and the view number v to generate a unique DRB round number rn . This can be achieved by concatenating b and v or using a function that maps the tuple $(b; v)$ to a unique integer value, for instance, a pairing function such as the Cantor pairing function.

Using this modified assignment for rn , we ensure that each view of a consensus round has a distinct DRB number, preventing pre-computation and replay attacks targeting beacon generation. This approach allows for the secure integration of DRB into the PBFT consensus process while maintaining the desirable security properties of both schemes.

B. Randomness Request

Using the beacon as a source of random numbers in smart contracts introduces security risks. It is possible for different transactions to receive the same random numbers, which makes the system vulnerable to duplicate transaction replay attacks. Adversaries can exploit this vulnerability by flooding the system with numerous transactions within the same block. Furthermore, if on-chain data is utilized to process the beacon, attackers could potentially precompute random numbers and incorporate them into their attack contracts, leading to precomputation attacks. Even if random numbers are generated securely, generating the same random numbers for multiple requests in a single transaction would result in duplicate requests and expose the system to pre-computation attacks.

To mitigate these concerns, BFTRAND.Request employs a pseudo-random function (PRF) to convert the beacon into multiple unique and unpredictable random numbers. We use the transaction hash as an identifier to ensure that transactions receive distinct random numbers and enter these numbers into the PRF. To prevent attackers from precomputing random numbers, the “GetRandom” function is designed to return different random numbers each time it is called. This is achieved by incorporating a global counter c as input, guaranteeing that each new request for random numbers has a unique input and produces a different output.

C. Mitigating PUAs

Ensuring secure RNP in BFTRAND requires addressing PUAs. We propose Input Validation-Based Detection (IVD) as

a means to counter PUAs without compromising transaction atomicity.

The IVD method uses existing smart contract regulations to authenticate user input, identify, and mitigate potential attacks. By individually examining each potential entry point for a PUA, the IVD approach ensures thoroughness in addressing security vulnerabilities. Furthermore, IVD maintains transaction atomicity by adhering to unaltered contract rules, making it extremely difficult for adversaries to exploit or bypass the IVD detection mechanism. Adversaries would need to create an entirely new PUA to circumvent the system, which poses a formidable challenge.

The IVD function takes two parameters: $\{SC; \mathcal{T}\}$, which represent the victim contract SC and the invocation of the transaction $\mathcal{T}$, and $addr$, the intended interaction address for the victim contract. If $\{SC; \mathcal{T}\}$ is not empty on *line 1*, IVD proceeds to PUA script, contract, and fee detection. At *Line 4*, two contract variables, $\$G_{min}$ and S_{max} , established during development, are retrieved. $\$G_{min}$ represents the minimum transaction fee required to cover all execution paths, while S_{max} indicates the maximum allowed transaction script size, which prevents appending the verification script. At *Line 6*, the maximum transaction fee $\$g_{max}$ and the transaction script S are extracted from the parameters. At *line 8*, IVD ensures the absence of fee PUA issues by verifying that the transaction fee is sufficient. At *Line 10*, Script PUA threats are checked by comparing the transaction size to the contract's maximum script size. If either check fails, IVD is reverted. Otherwise, IVD verifies direct transaction initiation of contract execution by obtaining the execution's entry script from the virtual machine at *Line 13*. If the acquired entry script and the transaction script are not identical, IVD is reverted. At *Line 18*, IVD retrieves the contract script for $addr$ from the blockchain. If the loaded script is nonempty, indicating a contract account and potential Fallback PUA, IVD is reverted.

VI. SECURITY ANALYSIS OF BFTRAND PROTOCOL

In this section, we provide a brief security analysis of the BFTRAND protocol, focusing on the four requirements (§III-C). The formal proofs can be found in the full version [9].

Theorem VI.1 (Pseudorandomness). *If the underlying DRB is $(f; k; n)$ -secure with $f \leq t < k \leq 2t + 1$, and PRF is a pseudorandom function, then BFTRAND satisfies pseudorandomness.*

Proof Sketch. This property follows directly from the pseudorandomness (i.e., the standard pseudorandomness described in [51]) of the underlying DRB protocol and PRF . Specifically, since the random beacons generated by DRB and the output from PRF are pseudorandom, the resulting random numbers are also pseudorandom. The full proof is provided in the full version [9].

Theorem VI.2 (Uniqueness). *BFTRAND satisfies uniqueness if PBFT is $(f; t; n)$ secure, DRB is $(f; k; n)$ secure, with $f \leq t < k \leq 2t + 1$, and PRF is a pseudorandom function.*

Proof Sketch. Since the adversary can control at most f nodes, as guaranteed by the underlying PBFT scheme, and this controlled number is less than the threshold defined in the DRB protocol, the adversary cannot generate valid random beacons at will, which is ensured by the uniqueness of the underlying DRB scheme. Furthermore, since the round number is determined by the block and view, only one predetermined beacon related to the block and view will be generated for each block. Additionally, as PRF is a deterministic algorithm, it guarantees that it can only produce the same result with the same inputs and that each random number generated from PRF has a different input. Consequently, every random number generated for smart contracts is unique. The full proof is in the full version [9].

Theorem VI.3 (Availability). *BFTRAND satisfies availability if PBFT is $(f; t; n)$ secure, DRB is $(f; k; n)$ secure and achieves uniqueness, for any f with $f \leq t < k \leq 2t + 1$.*

Proof Sketch. As BFTRAND receives at least $2t + 1$ beacons in the Finalize phase generated by different nodes, the beacon can be confirmed only if a value appears over t times. Following the uniqueness of DRB, the honest nodes generate the same beacons; hence, the honest nodes of BFTRAND can always output a valid beacon, even in the faulty nodes. Therefore, BFTRAND achieves *availability*. See the full proof in the full version [9].

Theorem VI.4 (Irreversibility). *BFTRAND achieves Irreversibility.*

Proof Sketch. BFTRAND verifies whether the contract is directly invoked by a transaction (*Contract PUA*), whether the interacting address is a contract account (*Fallback PUA*), whether there are adequate fees to cover all execution flows (*Fee PUA*), and whether there are additional transaction scripts (*Script PUA*). By eliminating and checking all conceivable methods for state detection and execution reverting at runtime, it ensures that user contracts execution cannot be maliciously reverted. Thus, *Irreversibility* is achieved. See the full proof in full version [9].

VII. IMPLEMENTATION AND EVALUATION

We have developed a prototype of BFTRAND using the Delegated Byzantine Fault Tolerant (dBFT) consensus algorithm [72], which is an optimized variant of PBFT [32] and is being adopted by many famous blockchain projects [13], [50], [67], [72]. We set its block time to 15 seconds. Since dBFT is a standard variation of PBFT, the protocols of dBFT can be adapted to other BFT platforms. In our implementation, we integrated BFTRAND into dBFT by modifying the messages in the classic phases, while preserving the original consensus function (§V-A). For the prototype DRB protocol BFTRAND, we instantiated the Boneh-Lynn-Shacham (BLS) threshold-based Dfinity-DRB [51], [55]. We use the neo virtual machine as the dApp execution environment, which can execute smart contracts that are implemented with C#.

Table I: Applications Transaction Fee (GAS/\$).

Method	Network Fee	System Fee
Loot::tokenURI	0.00593250/0.013	0.20694257/0.459
Neoverse::UnBoxing	0.00119552/0.002	0.07313472/0.162
Neoverse::BulkUnBoxing	0.00125752/0.002	0.36183988/0.803
RPS::Play	0.00616260/0.013	0.06588677/0.146

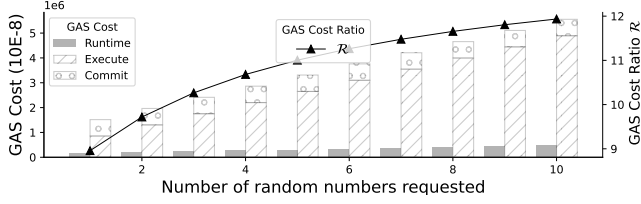


Figure 8: GAS cost when calling GetRandom. The left y-axis is the total GAS consumption, while the right y-axis is the GAS cost ratio $R = (\text{Commit} + \text{Execute})/\text{Runtime}$.

To showcase the advantages of BFTRAND, we implemented four applications: 1) Application for Fair NFT Distribution: We migrate the Loot [78] token distribution contract to nvm, where each bag is assigned a random rarity when users claim Loot tokens; 2) Application to Request for multiple random numbers: We reimplemented Neoverse [75] to generate multiple random values in a single transaction, allowing users to open multiple Blind Boxes simultaneously; 3) Application for Fair Gaming Outcome: We developed a rock-paper-scissor contract that enables users to participate in prize giveaways with random and fair outcomes; 4) Application for Performance Comparison: We designed an application to compare the performance of commit-execute random number generation solutions with BFTRAND.

We demonstrate the efficacy of BFTRAND and its superiority over other RNPs by comparing BFTRAND with them. We evaluated BFTRAND using a quad-core 3.6 GHz Intel(R) E3-1275 v5 CPU [57] and 32 GB of memory. The operating system used was Ubuntu 20.04 LTS with the Linux kernel version 5.4.0-91-generic, and the SDK version was .NET 6.0.1. The lines of code can be found in the full version [9].

Application Transaction Cost. Table I summarizes the transaction costs of running the developed applications. In nvm, the network fee is proportional to the length of the transaction script, while the OpCodes determine the system fee that a transaction executes. GAS is the token in nvm that pays the transaction fee. Currently, one GAS costs \$2.22, and \$1 buys 0.45 GAS. Loot::tokenURI provides a token with a randomly determined rarity to the user. Neoverse::UnBoxing purchases and opens one blind box, and Neoverse::BulkUnBoxing purchases and opens five blind boxes. RPS::Play produces a random shape and compares it with the one provided by the user. This evaluation demonstrates that smart contracts can conduct random number-related operations inexpensively with one single transaction.

Transaction Fee Cost. We compared the fees for requesting multiple random numbers between

BFTRAND and BFTRAND_{commit-execute}. In the case of BFTRAND_{commit-execute}, two transactions are needed to complete a request for random numbers. Therefore, we compare the cost of BFTRAND with the sum of the costs of two transactions in BFTRAND_{commit-execute}. The implementation of generating multiple random numbers with BFTRAND_{commit-execute} is carried out in C# on the VRF-based Chainlink platform¹. The evaluation result is shown in Figure 8.

From the results, we observed that the fee to request one random number with BFTRAND is 0:00169291 GAS (\$0.037), which is only 11% of the cost of issuing a random number request for BFTRAND_{commit-execute} (0:01516372 GAS, \$0.033). If every transaction requests ten random numbers, then BFTRAND saves 91.6% GAS (0.05088786 GAS, \$0.113). BFTRAND saves a considerable amount of transaction fees because BFTRAND_{commit-execute} pays transaction fees for two transactions, Commit and Execute. Execute needs to verify the fairness of the beacon and extend the beacon to random numbers in the smart contract through the virtual machine, which is expensive and inefficient. In contrast, BFTRAND generates random numbers in the native environment and guarantees the fairness of the beacon through consensus.

GAS Cost of Irreversibility Check. The cost to ensure irreversibility with IVD is less than 0:00001 GAS (\$0.00002), which is negligible compared to the transaction cost.

Blockchain Overhead. To evaluate the impact of RNP on the blockchain ledger, we define the size of the data that must be written on the blockchain to complete a request for random numbers as blockchain overhead (BO). Since Chainlink VRF [33] is one of the most influential RNP solutions, we use it as a baseline. Let the transaction size of $\mathcal{T}_{\text{execute}}$ be len . We evaluate the BO with the transaction size $\mathcal{T}_{\text{execute}}$ $len = 193$ bytes (the size of a typical nvm transfer transaction) and $len = 624$ bytes², while $\mathcal{T}_{\text{commit}}$ size is set as the nvm transaction size of 193 bytes. The BO for BFTRAND is 193 bytes, and the overhead for BFTRAND_{commit-execute} is $193 + len$. If $len = 193$ bytes, then BFTRAND saves approximately 50% BO compared to BFTRAND_{commit-execute}; If $len = 624$ bytes, BFTRAND saves approximately 76.4% BO.

Evaluation on Beacon Request. Since the adopted DRB exploits the BLS signature to generate the random beacon, we evaluated the aggregation time of σ . Figure 9 shows the aggregation time on the logarithmic scale under the k -out-of- n threshold mode, where the number of BLS nodes is n , and we set k to $\lceil (n+1)/2 \rceil$. When there are 7 consensus nodes (consistent with the neo-chain) and $k = 4$, the time cost of aggregating the BLS signatures is approximately 318 μ s; when the number of consensus nodes increases to 23 (comparable

¹Chainlink VRF [34] is the most influential RNP solution, which has processed approximately 8 million requests for random numbers for more than 656 projects.

²The size of a commit-execute transaction in Chainlink VRF [47], which includes the random number, proof, contract addresses, accounts, and other required parameters that cannot be ignored to verify the security properties of the random number generated from the blockchain.

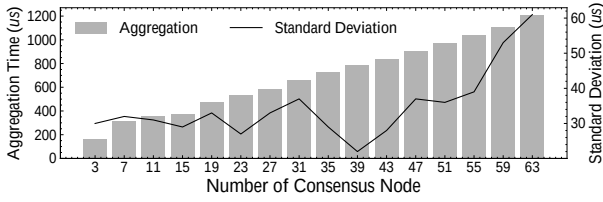


Figure 9: The BLS signature aggregation time.

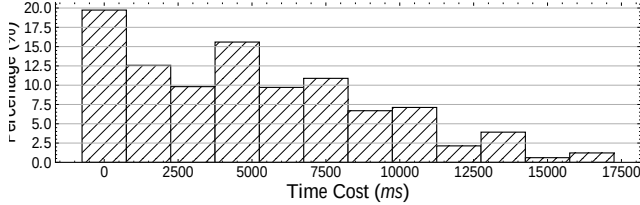


Figure 10: BLS setup time cost in a testnet with 22 consensus nodes. Note that BLS setup is separate to the consensus, it will not slow down the consensus.

to 21 nodes in EOS), the time cost is approximately 533 μ s. Adding BFTRAND to dBFT only incurs a negligible increase of 0.002% more time.

To evaluate the setup, we executed it on a private network of 22 dBFT nodes deployed on Azure. Azure nodes utilized Intel (R) Xeon (R) Platinum 8272CL CPU 2.6 GHz with Ubuntu18.04.6 LTS. The time to live (*ttl*) between the nodes was approximately 57 ms. The evaluation results are depicted in Figure 10 with 2;000 setups. More than half of the setups were completed in 5 seconds, with some cases (e.g. 15%) taking more than 10 seconds. This longer setup time is mainly influenced by the communication cost associated with collecting messages from all participants. Because the setup operation is independent of consensus and is only performed when the consensus committee requires an update and is separate from the consensus, execution of the setup does not impose overhead on the consensus layer.

RNP Solutions Comparison. In Table II, we provide a comprehensive comparison of BFTRAND with various projects related to the generation of random numbers. This includes beacon protocols [19], [30], [35], [54], [90], [90], blockchain projects with embedded random number protocols [2], [20]–[23], and oracles [34], [73].

- **Platform consensus** According to the table, the majority of the protocols in the comparison are based on the BFT consensus. This is because in order to generate random values in a distributed environment, the nodes must communicate interactively to establish consensus. Chainlink VRF, being an oracle, is platform independent and provides randomness for both BFT-based blockchains [27] and Proof-of-Work (PoW) based blockchains [47]. Automata [73] is a blockchain middleware specifically designed for Ethereum. As BFTRAND is built on top of the consensus algorithm PBFT, it can be easily adapted to any blockchain project that utilizes variants PBFT as its underlying consensus algorithm.

- **Method** To prevent a faulty node from obstructing the provision of random numbers, HDrand, RandRiper, Dfinity, Elrond and BFTRAND use distributed random beacon (DRB) protocols to generate random numbers. On the other hand, Klaytn and Harmony use verifiable random functions to ensure that the system generates deterministic random values. In addition to threshold signatures and verifiable random functions, several projects such as Secret, Chainlink VRF, and Automata use Trusted Execution Environments (TEE) [7], [11], [71], which are hardware-protected isolated execution environments, to generate random values in a secure and unbiased manner. The usage of TEE requires operating on devices that support TEE.

- **Comparing the number of random numbers.** We compare the ability of random number generators (RNPs) to generate random numbers. Random beacon projects such as RandHerd, RandHound, and BRandRiper focus solely on beacon generation, limiting the available random value to the number of beacons σ . On the other hand, Secret and Elrond provide built-in pseudorandom functions that can generate numerous random numbers by expanding σ . However, the random value they can supply is only upper bound by the consensus (e.g., GAS limit), represented as ∞ . In the case of BFTRAND, random numbers are generated using a pseudorandom function (PRF) of the consensus-based beacon. Therefore, the number of random numbers generated by BFTRAND is also constrained by consensus, such as the transaction fee.

- **Examining latency.** None of the random number generators used by random beacon or blockchain projects can generate random values within a single consensus round due to vulnerabilities to random number attacks. However, Automata stands out among the compared projects. It is the only one capable of providing run-time random numbers to smart contracts. Automata [73] is a trusted third-party middleware solution for the Ethereum Virtual Machine (EVM) platform. It generates random numbers in a Trusted Execution Environment (TEE) through Verifiable Random Functions (VRF) and uses Ethereum EIP712 [83] to encapsulate a user transaction and its requested random numbers within an EIP712 transaction.

While Automata is theoretically vulnerable to Pre-Unlock Attack (PUA), it is highly improbable in practice due to its closed solution nature, operating exclusively on its own services like NFTFair. On the contrary, BFTRAND is an RNP solution specifically designed for BFT-based open blockchain platforms, free from the reliance on specialized hardware and trusted third parties.

In summary, BFTRAND stands out as the only secure runtime random number provider for BFT-based blockchains.

VIII. RELATED WORK

In this section, we provide a concise overview of the existing literature on the provision of random numbers in blockchain systems, highlighting their limitations and vulnerabilities.

On-Chain RNPs and Commit-execute RNPs: On-chain RNP approaches utilize blockchain data as entropy sources [28], while commit-execute schemes involve a two-phase process

Table II: Comparison of RNPs for blockchain.

Protocol	Platform Consensus	Method(s)	Resistance (t)	# random values (r)	Latency (Consensus round)
Drand [30]	PABFT	Threshold SecretBLS	$t < n/2$	$\mathcal{O}(\sigma)$	≥ 2
HERB [35]	$\emptyset$	Threshold ElGamal	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
RandChain [54]	Sequential PoW	PoW	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
RandHerd [90]	BFT	Threshold Schnorr	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
RandHound [90]	BFT	Client based, PVSS	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
BRandRiper [19]	BFT	VSS, q-SDH	$t < n/2$	$\mathcal{O}(\sigma)$	≥ 2
Dfinity [2]	BFT	Threshold BLS	$t < n/2$	∞	≥ 2
Secret [23]	DPoS	Scrt-RNG, TEE	$t < n/2$	∞	≥ 2
Elrond [20]	Secure PoS	BLS, onchain data	$t < n/3$	∞	≥ 2
Klaytn [22]	Istanbul BFT	VRF	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
Harmony [21]	Fast BFT	VRF, VDF	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
*Chainlink VRF [34]	$\emptyset$	VRF, TEE	$t < n/2$	$\mathcal{O}(\sigma)$	≥ 2
*Automata [73]	$\emptyset$	VRF, TEE	$t < n/2$	∞	1
BFTRAND_{commit-execute}	BFT	$\emptyset$	$t < n/3$	$\mathcal{O}(\sigma)$	≥ 2
BFTRAND	BFT	Threshold BLS	$t < n/3$	∞	1 [§]

In the table, n denotes the number of consensus nodes, t is the maximum number of Byzantine nodes allowed in the system, and σ denotes the beacon. **Resistance** refers to the tolerance of the system for Byzantine faults. * is the off-chain third-party Oracle RNP. ∞ means the number of random numbers is upper-bounded by consensus. [§] BFTRAND is the first smart contract solution in runtime RNP on a BFT-based blockchain.

[2], [30], [60]. However, both methods have limitations, such as susceptibility to manipulation and prediction attacks [26], [40], [91], increased latency, and storage overhead [33], [49], [81].

Distributed Random Beacons and Smart Contract RNPs: The DRB protocols are designed to generate random numbers in a secure and decentralized manner [2], [20], [44]. Various studies on random beacons utilize different cryptographic primitives and threat models [18], [30], [52], [54], [60], [85], [86]. Some projects rely on homomorphic encryption [35], [76], while others operate in a permission-less setting [1], [4], [20], [23], [34], [43], [44], [48], [63]. While DRBs have made significant advancements in efficiently and securely generating random numbers, they are not explicitly designed as RNP solutions for smart contracts. Thus, they must be combined with the commit-execute scheme to provide randomness. However, the integration of DRB protocols with smart contracts at runtime presents unique security challenges.

Hardware RNPs and BFT Blockchain Projects: Hardware-based methods, such as ASIC-based VDF [24] and TEE-based oracles in Chainlink VRF [34] and Secret [23], rely on specific hardware. Several BFT blockchain projects have also been developed [5], [21], [22], [46], [56], [74], [79], [80], [88]. Our proposed BFT-based RNP solution has the potential to be integrated into these projects and is hardware independent.

BFTRAND is a secure and efficient RNP for smart contracts built on top of DRB schemes. It addresses the security challenges that arise when integrating DRBs into the blockchain for the provision of low-latency random numbers from smart contracts.

IX. CONCLUSION

Ensuring reliable randomness for smart contracts is a crucial aspect of various blockchain operations. Many decentralized applications heavily rely on randomness to achieve utility and security. However, existing commit-execute random number

provider protocols face challenges such as high costs on-chain and delayed processing times. To address these issues, this study introduces BFTRAND, a novel low-latency Random Number Protocol specifically designed for smart contracts on Byzantine Fault-Tolerant (BFT)-based blockchains. The proposed protocol, BFTRAND, is referred to as low latency due to its ability to securely provide random numbers using a single round of consensus, as opposed to the two rounds required by commit-execute RNPs. To assess its performance, we implemented a prototype of BFTRAND using a distributed random beacon protocol and seamlessly integrated it into the consensus layer without compromising efficiency. Furthermore, BFTRAND has been rigorously tested and proven secure against random number attacks. Furthermore, we identify various types of *post-reveal Undo Attacks* and propose effective mitigation strategies to improve the security of BFTRAND. Consequently, BFTRAND is the firstgisesandfod

- [10] Andreas M Antonopoulos and Gavin Wood. *Mastering ethereum: building smart contracts and dapps*. O'reilly Media, 2018.
- [11] ARM. Arm trustzone technology. <https://developer.arm.com/ip-products/security-ip/trustzone>, 2019-12-13.
- [12] Avi Asayag, Gad Cohen, Ido Grayevsky, Maya Leshkowitz, Ori Rottenstreich, Ronen Tamari, and David Yakira. A fair consensus protocol for transaction ordering. In *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pages 55–65. IEEE, 2018.
- [13] Diem Association. Diem. <https://www.diem.com/en-us/>, 2019.
- [14] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. A survey of attacks on ethereum smart contracts (sok). *IACR Cryptology ePrint Archive*, 2017:1–35, 2017.
- [15] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. Rbft: Redundant byzantine fault tolerance. In *2013 IEEE 33rd International Conference on Distributed Computing Systems*, pages 297–306. IEEE, 2013.
- [16] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. Consensus in the age of blockchains. *arXiv preprint arXiv:1711.03936*, 2017.
- [17] Alysson Bessani, João Sousa, and Eduardo EP Alchieri. State machine replication for the masses with bft-smart. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 355–362. IEEE, 2014.
- [18] Adithya Bhat, Aniket Kate, Kartik Nayak, and Nibesh Shrestha. Op-trand: Optimistically responsive distributed random beacons. *Cryptology ePrint Archive*, 2022.
- [19] Adithya Bhat, Nibesh Shrestha, Zhongtang Luo, Aniket Kate, and Kartik Nayak. Randpipe—reconfiguration-friendly random beacons with quadratic communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3502–3524, 2021.
- [20] Elrond Blockchain. Random numbers in smart contracts. <https://docs.elrond.com/developers/developer-reference/random-numbers-in-smart-contracts/>, 2022.
- [21] Harmony Blockchain. Harmony randomness. <https://docs.harmony.one/home/general/technology/randomness>, 2022.
- [22] Klaytn Blockchain. Consensus randomness. <https://docs.klaytn.com/klaytn/design/consensus-mechanism>, 2022.
- [23] Secret Blockchain. Secret randomness. <https://docs.scrtnetwork.dev/developing-secret-contracts.html#randomness>, 2022.
- [24] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In *Annual international cryptology conference*, pages 757–788. Springer, 2018.
- [25] Joseph Bonneau, Andrew Miller, Jeremy Clark, Arvind Narayanan, Joshua A Kroll, and Edward W Felten. On the necessity of a prescribed block validity consensus: Analyzing bitcoin unlimited mining protocol. *Queue*, 13(7):20, 2015.
- [26] Joseph Bonneau, Andrew Miller, Jeremy Clark, Arvind Narayanan, Joshua A Kroll, and Edward W Felten. Sok: Research perspectives and challenges for bitcoin and cryptocurrencies. *2015 IEEE Symposium on Security and Privacy*, pages 104–121, 2015.
- [27] BscScan. Vrfcoordinator. <https://bscscan.com/address/0x747973a5A2a4Ae1D3a8fDF5479f1514F65Db9C31#analytics>, 2022.
- [28] Vitalik Buterin and ethereum.org. A next-generation smart contract and decentralized application platform. *white paper*, 3(37), 2014.
- [29] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*, pages 524–541. Springer, 2001.
- [30] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constantinople: Practical asynchronous byzantine agreement using cryptography. *Journal of Cryptology*, 18(3):219–246, 2005.
- [31] Christian Cachin and Marko Vukolić. Blockchain consensus protocols in the wild. *arXiv preprint arXiv:1707.01873*, 2017.
- [32] Miguel Castro, Barbara Liskov, et al. Practical byzantine fault tolerance. In *OsDI*, volume 99, pages 173–186, 1999.
- [33] chain.link. <https://chain.link/>. <https://chain.link/>. accessed: 2020-08-18.
- [34] Chainlink. Get a random number. <https://docs.chain.link/docs/get-a-random-number/>, 2021.
- [35] Alisa Cherniaeva, Iliia Shirobokov, and Omer Shlomovits. Homomorphic encryption random beacon. *Cryptology ePrint Archive*, 2019.
- [36] Usman W Chohan. Non-fungible tokens: Blockchains, scarcity, and value. *Critical Blockchain Research Initiative (CBRI) Working Papers*, 2021.
- [37] Allen Clement, Edmund L Wong, Lorenzo Alvisi, Michael Dahlin, and Mirco Marchetti. Making byzantine fault tolerant systems tolerate byzantine faults. In *NSDI*, volume 9, pages 153–168, 2009.
- [38] coinmarketcap.com. Today's cryptocurrency prices by market cap. <https://coinmarketcap.com/>, March 2022.
- [39] Mauro Conti, Divya Diwaker, Sankardas Roy, and Radha Poovendran. A survey on blockchain: Techniques, applications, and performance evaluation. *IEEE Communications Surveys & Tutorials*, 21(4):2678–2716, 2019.
- [40] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges. *arXiv preprint arXiv:1904.05234*, 2019.
- [41] DAMIEN. Hackers targeted eosbet dapp, \$200k worth of eos stolen, 2018.
- [42] Sourav Das, Vinith Krishnan, Irene Miriam Isaac, and Ling Ren. Spurt: Scalable distributed randomness beacon with transparent setup. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2502–2517. IEEE, 2022.
- [43] TRON Core Devs. Safe practice of tron solidity smart contracts: Implement random numbers in the contracts, Mar 2020.
- [44] Drand. Drand/drand: a distributed randomness beacon daemon - go implementation. <https://github.com/drand/drand>.
- [45] Chainlink Ecosystem. Chainlink ecosystem. <https://www.chainlinkecosystem.com/ecosystem>, 2022.
- [46] EOSIO. Eosio website. <https://eos.io/>, November 2022.
- [47] Etherscan. Chainlink vrf. <https://etherscan.io/address/0x271682DEB8C4E0901D1a1550aD2e64D568E69909/>, 2022.
- [48] Fetch.ai. Launching our random number beacon on binance smart chain. <https://medium.com/fetch-ai/launching-our-random-number-beacon-on-binance-smart-chain-8e3b7aa52be6>, Oct 2020.
- [49] Filecoin. Collaboration with the ethereum foundation on vdfs. <https://filecoin.io/blog/posts/collaboration-with-the-ethereum-foundation-on-vdfs/>, 2019.
- [50] The Linux Foundation. Hyperledger fabric. <https://www.hyperledger.org/use/fabric>, 2016.
- [51] David Galindo, Jia Liu, Mihair Ordean, and Jin-Mann Wong. Fully distributed verifiable random functions and their application to decentralised random beacons. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 88–102. IEEE, 2021.
- [52] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 295–310. Springer, 1999.
- [53] Ilya Grishchenko, Matteo Maffei, and Clara Schneidewind. A semantic framework for the security analysis of ethereum smart contracts. In *International Conference on Principles of Security and Trust*, pages 243–269. Springer, 2018.
- [54] Runchao Han, Jiangshan Yu, and Haoyu Lin. Randchain: Decentralised randomness beacon from sequential proof-of-work. *IACR Cryptol. ePrint Arch.*, 2020:1033, 2020.
- [55] Timo Hanke, Mahnush Movahedi, and Dominic Williams. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548*, 2018.
- [56] Helium.com. Helium documentation. <https://docs.helium.com/blockchain/consensus-protocol/>, 2022.
- [57] Intel. Intel® xeon® processor e3 v5 family. <https://ark.intel.com/content/www/us/en/ark/products/88177/intel-xeon-processor-e3-1275-v5-8m-cache-3-60-ghz.html>, 2019-12-3.
- [58] Aljosha Judmayer, Nicholas Stifter, Philipp Schindler, and Edgar Weippl. Estimating (miner) extractable value is hard, let's go shopping! *Cryptology ePrint Archive*, 2021.
- [59] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. Zeus: Analyzing safety of smart contracts. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2018.
- [60] John Kelsey, Luís TAN Brandão, Rene Peralta, and Harold Booth. A reference for randomness beacons: Format and protocol version 2. Technical report, National Institute of Standards and Technology, 2019.
- [61] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 583–598. IEEE, 2018.

- [62] Aashish Kolluri, Ivica Nikolic, Ilya Sergey, Aquinas Hobor, and Prateek Saxena. Exploiting the laws of order in smart contracts. In *Proceedings of the 28th USENIX Security Symposium*, pages 1317–1334. USENIX Association, 2019.
- [63] Mikhail Krasnoselskii, Grigorii Melnikov, and Yury Yanovich. Distributed random number generator on hedera hashgraph. In *2020 the 3rd International Conference on Blockchain Technology and Applications*, pages 7–11, 2020.
- [64] Jae Kwon. Tendermint: Consensus without mining. *Draft v. 0.6, fall*, 1(11), 2014.
- [65] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. In *Concurrency: the works of leslie lamport*, pages 203–226. 2019.
- [66] Kfir Lev-Ari, Alexander Spiegelman, Idit Keidar, and Dahlia Malkhi. Fairledger: A fair blockchain protocol for financial institutions. *arXiv preprint arXiv:1906.03819*, 2019.
- [67] Ontology Foundation Ltd. Ontology. <https://ont.io/>, 2018.
- [68] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 254–269. ACM, 2016.
- [69] J-P Martin and Lorenzo Alvisi. Fast byzantine consensus. *IEEE Transactions on Dependable and Secure Computing*, 3(3):202–215, 2006.
- [70] Sky Mavis. Paid whitepaper. <https://docsend.com/view/jdbdpza9d9nehf2>, January 2021.
- [71] Frank McKeen, Ilya Alexandrovich, Alex Berenzon, Carlos V Rozas, Hisham Shafi, Vedvyas Shanbhogue, and Uday R Savagaonkar. Innovative instructions and software model for isolated execution. In *HASP@ISCA*, page 10, 2013.
- [72] neo.org. Neo smart economy. <https://neo.org/>, 2022.
- [73] Automata Network. Automata network. <https://www.ata.network>, 2021.
- [74] THETA NETWORK. Theta network. <https://docs.thetatoken.org/docs/whitepapers>.
- [75] NGD. Neoverse, 2022.
- [76] Thanh Nguyen-Van, Tuan Nguyen-Anh, Tien-Dat Le, Minh-Phuoc Nguyen-Ho, Tuong Nguyen-Van, Nhat-Quang Le, and Khuong Nguyen-An. Scalable distributed random number generation based on homomorphic encryption. In *2019 IEEE International Conference on Blockchain (Blockchain)*, pages 572–579. IEEE, 2019.
- [77] Ivica Nikolic, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. Finding the greedy, prodigal, and suicidal contracts at scale. In *Proceedings of the 34th Annual Computer Security Applications Conference*, pages 653–663. ACM, 2018.
- [78] Loot Project. Loot contract source code. <https://etherscan.io/address/0xff9c1b15b16263c61d017ee9f65c50e4ae0113d7#code>, 2021.
- [79] The Celo Project. Celo randomness: Celo docs. <https://docs.celo.org/celo-codebase/protocol/identity/randomness>.
- [80] The Kadena Project. Kadena whitepaper. <https://docs.kadena.io/basics/whitepapers/overview>.
- [81] Youcai Qian. Randao: Verifiable random number generation, 2017.
- [82] Mayank Raikwar and Danilo Gligoroski. Sok: Decentralized randomness beacon protocols. *arXiv preprint arXiv:2205.13333*, 2022.
- [83] Leonid Logvinov Remco Bloemen. Eip-712: Ethereum typed structured data hashing and signing. <https://eips.ethereum.org/EIPS/eip-712>.
- [84] Philipp Schindler, Aljosha Judmayer, Markus Hittmeir, Nicholas Stifter, and Edgar Weippl. Randrunner: Distributed randomness from trapdoor vdfs with strong uniqueness. 2021.
- [85] Philipp Schindler, Aljosha Judmayer, Nicholas Stifter, and Edgar Weippl. Hydrand: Efficient continuous distributed randomness. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 73–89. IEEE, 2020.
- [86] Srdan Daniel Simić, Robert Šajina, Nikola Tanković, and Darko Etinger. A review on generating random numbers in decentralised environments. In *2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO)*, pages 1668–1673. IEEE, 2020.
- [87] SlowMist. Slowmist hacked statistics. <https://hacked.slowmist.io/en/statistics/?c=all&d=all>, 2021.
- [88] solana.com. Solana documentation. <https://docs.solana.com/introduction>, 2022.
- [89] Stefan Stankovic. What is mev? ethereum’s invisible tax explained. <https://cryptobriefing.com/what-is-mev-ethereums-invisible-tax-explained/>, 2021.
- [90] Ewa Syta, Philipp Jovanovic, Eleftherios Kokoris Kogias, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Michael J Fischer, and Bryan Ford. Scalable bias-resistant distributed randomness. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 444–460. Ieee, 2017.
- [91] RANDAO Team. Randao: A dao working as rng of ethereum. <https://github.com/randao/randao>, 2016.
- [92] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, and Lau Cheuk Lung. Spin one’s wheels? byzantine fault tolerance with a spinning primary. In *2009 28th IEEE International Symposium on Reliable Distributed Systems*, pages 135–144. IEEE, 2009.
- [93] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung, and Paulo Verissimo. Efficient byzantine fault-tolerance. *IEEE Transactions on Computers*, 62(1):16–30, 2011.
- [94] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. In *Ethereum project yellow paper*, volume 151, pages 1–32, 2014.
- [95] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 347–356, 2019.